Assignement 1 :

The **DBpedia Endpoint** is an online service that allows users to query the **DBpedia knowledge graph** using the **SPARQL query language**.

DBpedia is a large, structured knowledge base created by extracting information from Wikipedia and representing it as **RDF triples** (subject–predicate–object)

Why is the DBpedia Endpoint useful?

The DBpedia Endpoint is useful because it allows applications to:

- Retrieve structured information from Wikipedia.
- Explore relationships between entities.
- Build semantic web and linked data applications.
- Answer complex questions using SPARQL instead of manually reading Wikipedia pages.

Line 1

```
PREFIX dbr: <http://dbpedia.org/resource/>
```

What does it do?

This line defines a **prefix** (a shortcut).

Instead of writing the full URI every time:

```
http://dbpedia.org/resource/Jennifer_Aniston
```

we can simply write:

```
dbr:Jennifer_Aniston
```

Here:

- **dbr** stands for **DBpedia Resource**
- It refers to actual entities (people, places, movies, companies, etc.)

For example:

```
dbr:Jennifer_Aniston
dbr:Brad_Pitt
dbr:Italy
dbr:Google
```

These are all **resources** in DBpedia.

Line 2

```
PREFIX dbo: <http://dbpedia.org/ontology/>
```

This defines another shortcut.

Instead of writing

```
http://dbpedia.org/ontology/spouse
```

we simply write

```
dbo:spouse
```

Here:

- **dbo** stands for **DBpedia Ontology**
- It contains properties (relationships or attributes).

Examples include:

```
dbo:spouse
dbo:birthPlace
dbo:author
dbo:director
dbo:capital
```

These describe relationships between resources.

A simple way to remember it is:

- **dbr = Things**
- **dbo = Relationships**

Line 3

- `SELECT ?spouse`
- This tells SPARQL:
- "I want the value stored in the variable `?spouse`."
- The question mark (`?`) indicates a **variable**.
- Just like a variable in Python:
- `x`
- In SPARQL we write:
- `?spouse`
- This variable will hold the answer.

- # Line 4
- `WHERE`
- This introduces the search condition.
- It means:
- "Find data that satisfies the following pattern."
- # Line 5
- `{`
- This begins the graph pattern (the search conditions).
- # The most important line
- `dbr:Jennifer_Aniston dbo:spouse ?spouse .`
- This line is an **RDF Triple Pattern**.
- It has exactly three parts:

Subject	Predicate	Object
dbr:Jennifer_Aniston	dbo:spouse	?spouse

- Let's examine each one.

Query 2 :

```
PREFIX dbr: <http://dbpedia.org/resource/>

PREFIX dbo: <http://dbpedia.org/ontology/>


SELECT (COUNT(?film) AS ?numFilms) WHERE {

 ?film dbo:starring dbr:Jennifer_Aniston .

}
```

`PREFIX dbr:`

- Defines the prefix for DBpedia resources (entities), such as `Jennifer_Aniston`.

`PREFIX dbo:`

- Defines the prefix for DBpedia ontology properties, such as `starring`.

`SELECT (COUNT(?film) AS ?numFilms)`

- Instead of returning every matching film, this query counts them.
- `COUNT(?film)` counts the number of films found.
- `AS ?numFilms` stores the result in a variable named `?numFilms`.

```
WHERE {
```

- Starts the graph pattern that specifies what to search for.

```
?film dbo:starring dbr:Jennifer_Aniston .
```

- `?film` is a variable representing any film.
- `dbo:starring` is the relationship meaning "features an actor."
- `dbr:Jennifer_Aniston` is the specific actor we are searching for.
- In other words, the query looks for every film in DBpedia where Jennifer Aniston is listed as a starring actor.

```
}
```

- Ends the `WHERE` clause.

Query 3 :

5. Give me the total number of actors that performed in the latest film of your favorite actor.

```
PREFIX dbr: <http://dbpedia.org/resource/>

PREFIX dbo: <http://dbpedia.org/ontology/>


SELECT (COUNT(?actor) AS ?numActors) WHERE {

  ?film dbo:starring dbr:Jennifer_Aniston ;

      dbo:releaseDate ?date ;

      dbo:starring ?actor .

}

ORDER BY DESC(?date)
```

- **`PREFIX dbr:`** Defines the prefix for DBpedia resources, such as `Jennifer_Aniston`.
- **`PREFIX dbo:`** Defines the prefix for DBpedia ontology properties.

- **`SELECT (COUNT(?actor) AS ?numActors)`** Counts the number of actors and stores the result in the variable `?numActors`.
- **`?film dbo:starring dbr:Jennifer_Aniston`** Finds films in which Jennifer Aniston starred.
- **`dbo:releaseDate ?date`** Retrieves the release date of each film.
- **`dbo:starring ?actor`** Retrieves every actor who starred in each film.
- **`ORDER BY DESC(?date)`** Sorts the films from newest to oldest.

Limist 1 : **Keep only the first result**, which is the **most recently released film**.

So the intended logic becomes:

1. Find all films starring Jennifer Aniston.
2. Retrieve the release date of each film.
3. Sort the films by release date in descending order (newest first).
4. **Keep only the latest film** (`LIMIT 1`).
5. Count the actors who starred in that film.

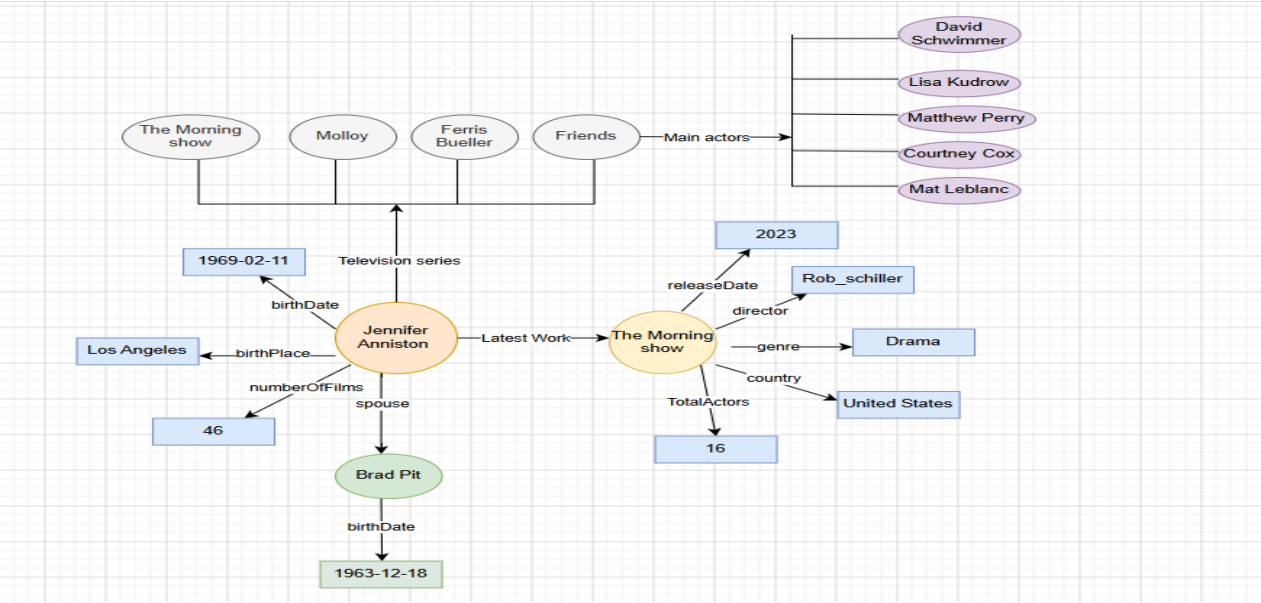
Explaining the RDF Graph :

Your graph is actually well structured. It follows the RDF idea of:

Subject → Predicate → Object

where:

- **Subject** = the main entity (circle)
- **Predicate** = the relationship (arrow label)
- **Object** = another entity or a literal value



- **The circles represent entities (or nodes) in the knowledge graph, such as people and TV series. The rectangles represent literal values or attributes, such as dates, numbers, genres, and countries. The arrows represent relationships (predicates) between nodes, and the labels on the arrows indicate the type of relationship, such as *spouse*, *birthPlace*, or *director*. Overall, the graph follows the RDF structure of Subject–Predicate–Object, where entities are connected through semantic relationships or linked to their attribute values.**